

Umurava AI Hackathon

AI-Powered Talent Profile Screening Tool

Software Requirements Specification (SRS)

Prepared by Mr. Saveur

Page - 1

TABLE OF CONTENT

1. INTRODUCTION	3
1.1. PURPOSE	3
1.2. DOCUMENT CONVENTIONS	3
1.4. PROJECT SCOPE	4
2. PROBLEM STATEMENT	5
3. SYSTEM FEATURES	6
3.1 RECRUITER DASHBOARD	6
3.2 JOB MANAGEMENT	7
3.3 APPLICANT INGESTION	8
3.4 AI SCREENING ENGINE	9
3.5 SHORTLIST & RESULTS VIEW	10
3.6 AI REASONING & EXPLAINABILITY	11
4. SYSTEM ARCHITECTURE	12
5. TECHNOLOGY STACK	13
6. DEPLOYMENT REQUIREMENTS	14
7. TEAM COMPOSITION	15
8. REFERENCES	16

1. INTRODUCTION

1.1. PURPOSE

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements for an AI-powered talent profile screening tool to be built as part of the Umurava AI Hackathon. The challenge theme is: "An innovation challenge to build AI Products for the Human Resources Industry."

This document serves as the technical blueprint for the participating development team, outlining what the system must do, how it must behave, and the standards it must meet. The resulting product is intended to assist recruiters in efficiently screening and shortlisting job applicants while preserving human-led final hiring decisions.

1.2. DOCUMENT CONVENTIONS

Terminology

- **Recruiter:** Refers to the primary user of the system — an HR professional or hiring manager who interacts with the recruiter-facing interface.
- **Applicant:** Refers to a job candidate whose profile or resume is submitted into the system for screening.
- **Admin Panel:** Refers to the dashboard interface used by the recruiter to manage jobs, applicants, and screening results.
- **AI Screening Engine:** Refers to the Gemini-powered component responsible for analyzing, scoring, and ranking applicants.

Abbreviations

- **UI:** User Interface
- **UX:** User Experience
- **SRS:** Software Requirements Specification
- **LLM:** Large Language Model
- **API:** Application Programming Interface
- **CSV:** Comma-Separated Values
- **HR:** Human Resources
- **KPI:** Key Performance Indicator

Formatting Conventions

- **Bold:** Used for section headings and significant terms on first use.
- *Italic:* Used to indicate examples and non-functional requirements.
- **Code Formatting:** Inline code snippets or configuration references are represented in monospace font (e.g., `.env`, `schema.json`).

References

- References to external tools or APIs (e.g., Gemini API, MongoDB, Next.js) are made explicitly, with documentation links provided in the references section.

1.4. PROJECT SCOPE

In-Scope

The following features and functionalities will be included in the Umurava AI Hackathon project:

1. Recruiter Dashboard: A unified interface for managing job postings, applicant data, and screening results.
2. Job Management: Create, edit, and view job postings with required skills and experience criteria.
3. Applicant Ingestion: Support ingestion from two sources — structured Umurava talent profiles and external CSV/resume uploads.
4. AI Screening Engine: Gemini API-powered scoring and ranking of applicants against job requirements.
5. Shortlist View: Display of ranked Top 10 or Top 20 candidates with match scores.
6. AI Reasoning: Natural-language explanations per shortlisted candidate covering strengths, gaps, and relevance.
7. Deployment: Fully deployed and accessible production-grade application.

Out-of-Scope

The following are beyond the scope of this project:

8. Automated offer-letter generation or contract management.
9. Candidate-facing portal or self-service application submission.
10. Full ATS (Applicant Tracking System) functionality beyond what is specified.
11. Multi-language support — the system will operate in English only.
12. Complex HR payroll or onboarding workflows.

2. PROBLEM STATEMENT

Recruiters today face two major challenges:

- High application volumes that significantly increase time-to-hire.
- Difficulty objectively comparing candidates across diverse profiles and formats.

The core problem is:

"How can AI be used to accurately, transparently, and efficiently screen and shortlist job applicants across both structured talent profiles and unstructured resumes while preserving human-led hiring decisions?"

The system to be built must:

- Understand job requirements and ideal candidate profiles.
- Analyze multiple applicants at once.
- Produce a ranked shortlist (Top 10 or Top 20).
- Clearly explain why each candidate was shortlisted — covering strengths, gaps, and relevance.

3. SYSTEM FEATURES

3.1 RECRUITER DASHBOARD

3.1.1 Description and Priority

The recruiter dashboard serves as the central hub for all recruiter activities. It provides an overview of active job postings, pending screenings, and shortlist results in one unified interface.

Priority: High

3.1.2 Stimulus/Response Sequences

Stimulus: The recruiter logs in to the dashboard.

Response: The system validates credentials and displays the dashboard home with summary statistics: total jobs, total applicants, pending screenings, and recent shortlists.

Stimulus: The recruiter navigates to a job from the dashboard.

Response: The system loads the job detail page showing job description, applicant count, and screening status.

Stimulus: The recruiter triggers an AI screening from the dashboard.

Response: The system initiates the AI engine and displays a real-time progress indicator while processing applicants.

3.1.3 Non-Functional Requirements

Requirement	Description
REQ-1	Dashboard must load within 3 seconds on standard internet connection.
REQ-2	All dashboard statistics must reflect real-time data from the database.
REQ-3	The dashboard must be responsive on desktop and tablet viewports.
REQ-4	Support dark mode and light mode toggle.

3.2 JOB MANAGEMENT

3.2.1 Description and Priority

This feature allows recruiters to create and manage job postings. Each job includes role details, required skills, experience level, and ideal candidate criteria that the AI engine will use for screening.

Priority: High

3.2.2 Stimulus/Response Sequences

Stimulus: The recruiter creates a new job posting with role, requirements, skills, and experience.

Response: The system saves the job to the database and makes it available for applicant ingestion and AI screening.

Stimulus: The recruiter edits an existing job posting.

Response: The system updates the job details in real-time and flags any previously completed screenings as outdated.

Stimulus: The recruiter deletes a job posting.

Response: The system removes the job and all associated applicant and screening data after confirmation.

Stimulus: The recruiter views the list of all job postings.

Response: The system displays jobs with status indicators (Open, Screening, Shortlisted, Closed), creation date, and applicant count.

3.2.3 Non-Functional Requirements

Requirement	Description
REQ-1	Job creation form must validate all required fields before submission.
REQ-2	Support tagging jobs by category or department.
REQ-3	Enable filtering and sorting of job list by date, status, and applicant count.
REQ-4	Job data must persist in MongoDB with proper schema validation.

3.3 APPLICANT INGESTION

3.3.1 Description and Priority

This feature handles the ingestion of applicant data from two distinct sources: structured talent profiles from the Umurava platform, and unstructured external data via CSV spreadsheets and resume uploads (PDF/links). The system must parse and normalize the data before passing it to the AI engine.

Priority: High

3.3.2 Stimulus/Response Sequences

Scenario 1 — Umurava Platform Profiles

Stimulus: The recruiter selects a job and loads applicant profiles from the Umurava talent pool.

Response: The system fetches and displays structured profiles that conform to the Umurava Talent Profile Schema.

Stimulus: The recruiter reviews the loaded profiles before triggering screening.

Response: The system shows a summary table of all applicants including name, skills, and experience years.

Scenario 2 — External Job Board Applicants

Stimulus: The recruiter uploads a CSV or Excel spreadsheet of applicant data.

Response: The system parses the file, maps columns to the internal applicant schema, and displays the parsed results for review.

Stimulus: The recruiter uploads PDF resumes or provides resume links.

Response: The system extracts text from the PDFs/links and structures the parsed data into candidate profiles.

Stimulus: Parsing encounters an unrecognized format or missing fields.

Response: The system highlights the problematic entries and prompts the recruiter to manually fill in or correct the data.

3.3.3 Non-Functional Requirements

Requirement	Description
REQ-1	Dummy profile data must strictly conform to the Umurava Talent Profile Schema.
REQ-2	CSV ingestion must support common delimiters and handle missing columns gracefully.
REQ-3	PDF resume parsing must extract at minimum: name, skills, education, and experience.
REQ-4	Ingestion pipeline must handle a minimum of 50 applicants per job without timeout.

3.4 AI SCREENING ENGINE

3.4.1 Description and Priority

This is the core feature of the application. The AI Screening Engine uses the Gemini API (mandatory) to analyze all applicants against a given job's requirements, assign weighted scores, and produce a ranked shortlist. Prompt engineering must be intentional and documented.

Priority: High

3.4.2 Stimulus/Response Sequences

Stimulus: The recruiter triggers AI screening for a job with loaded applicants.

Response: The system constructs a structured prompt containing the job requirements and all applicant profiles, then sends a request to the Gemini API.

Stimulus: The Gemini API processes the applicants and returns a ranked result.

Response: The system parses the AI response, stores the screening results in MongoDB, and updates the job status to 'Shortlisted'.

Stimulus: The AI engine encounters an API error or timeout.

Response: The system displays an error notification and allows the recruiter to retry without losing previously ingested applicant data.

Stimulus: The recruiter re-runs screening after updating the job criteria.

Response: The system marks the previous result as archived and produces a fresh shortlist based on the updated criteria.

3.4.3 Non-Functional Requirements

Requirement	Description
REQ-1	Gemini API must be the mandatory underlying LLM for all AI operations.
REQ-2	Scoring must apply weighted criteria: skills match, experience, education, and role relevance.
REQ-3	AI prompts must be documented with assumptions and limitations in the README.
REQ-4	AI outputs must be clean, structured, and recruiter-friendly (no raw JSON to end user).
REQ-5	Multi-candidate evaluation must be supported in a single prompt call where feasible.

3.5 SHORTLIST & RESULTS VIEW

3.5.1 Description and Priority

This feature displays the AI-generated ranked shortlist to the recruiter in a clear, visual format. The recruiter can view the top candidates, compare their scores, and access detailed reasoning for each.

Priority: High

3.5.2 Stimulus/Response Sequences

Stimulus: The recruiter navigates to the shortlist view for a screened job.

Response: The system displays a ranked table of Top 10 or Top 20 candidates with rank position, name, match score (0–100), and a recommendation tag.

Stimulus: The recruiter clicks on a shortlisted candidate.

Response: The system opens a detailed candidate panel showing full profile data, AI-generated strengths, gaps, and hiring recommendation.

Stimulus: The recruiter filters the shortlist by score range or recommendation tag.

Response: The system filters the displayed results in real-time without re-running the AI engine.

Stimulus: The recruiter exports the shortlist.

Response: The system generates a downloadable CSV or PDF report of the ranked shortlist with AI summaries.

3.5.3 Non-Functional Requirements

Requirement	Description
REQ-1	Shortlist must support configurable display of Top 10 or Top 20 candidates.
REQ-2	Rank, score, strengths, gaps, and recommendation must be clearly visible per candidate.
REQ-3	Results must render correctly on both desktop and tablet viewports.
REQ-4	Export feature must produce a properly formatted, downloadable document.

3.6 AI REASONING & EXPLAINABILITY

3.6.1 Description and Priority

Explainability is a core requirement of this system. Every shortlisted candidate must be accompanied by a natural-language explanation generated by the AI engine. This ensures that recruiters understand the basis of every recommendation and can defend their final hiring decisions.

Priority: High

3.6.2 Stimulus/Response Sequences

Stimulus: The recruiter views a shortlisted candidate's detail page.

Response: The system displays structured AI-generated reasoning including: candidate rank, match score, listed strengths, identified gaps or risks, and a final hiring recommendation.

Stimulus: The recruiter requests a comparison between two shortlisted candidates.

Response: The system displays a side-by-side comparison of scores and AI reasoning for both candidates.

Stimulus: The AI reasoning output contains vague or insufficient explanation.

Response: The system flags the candidate with a low-confidence indicator and suggests the recruiter review the profile manually.

3.6.3 Non-Functional Requirements

Requirement	Description
REQ-1	Every shortlisted candidate must include: rank, match score, strengths, gaps, and recommendation.
REQ-2	Reasoning must be written in clear, recruiter-friendly natural language.
REQ-3	AI assumptions and limitations must be documented in the project README.
REQ-4	The system must never make a final hiring decision — final authority rests with the human recruiter.

4. SYSTEM ARCHITECTURE

The system follows a three-tier architecture composed of a frontend layer, a backend API layer, and an AI orchestration layer, all connected to a MongoDB database.

4.1 Frontend (Next.js)

- Recruiter dashboard with job management and applicant views.
- Job input forms and applicant upload interfaces.
- Shortlist visualization and AI reasoning display.
- State management via Redux + Redux Toolkit.
- Styling via Tailwind CSS.

4.2 Backend (Node.js + TypeScript)

- REST API layer exposing endpoints for jobs, applicants, and screening results.
- Job and applicant data processing and validation.
- AI orchestration logic — constructing prompts and handling Gemini API responses.
- Authentication and authorization middleware.

4.3 AI Layer (Gemini API — Mandatory)

- Job-to-candidate matching using structured prompts.
- Candidate scoring and weighted ranking.
- Natural-language reasoning generation per shortlisted candidate.

4.4 Database (MongoDB)

- Collections: Jobs, Applicants, ScreeningResults.
- Hosted on MongoDB Atlas for production deployment.
- Schema validation enforced at the application layer.

5. TECHNOLOGY STACK

Layer	Technology
Language	TypeScript
Frontend	Next.js
State Management	Redux + Redux Toolkit
Styling	Tailwind CSS
Backend	Node.js
Database	MongoDB (Atlas)
AI / LLM	Gemini API (Mandatory)
Frontend Hosting	Vercel
Backend Hosting	Railway / Render / Fly.io

6. DEPLOYMENT REQUIREMENTS

The application must be deployed and publicly accessible online before the hackathon submission deadline.

6.1 Hosting Providers

- Vercel — Frontend (Next.js)
- Railway / Render / Fly.io — Backend (Node.js API)
- MongoDB Atlas — Database

6.2 Deployment Expectations

- A live public URL must be shared at submission.
- All environment variables must be securely configured (no hardcoded secrets in the codebase).
- Basic error handling must be in place for production (404 pages, network errors, API failures).
- The README must include setup instructions, environment variable documentation, and the AI decision flow explanation.

7. TEAM COMPOSITION

Each team must have 2 to 5 members. The minimum required roles are defined below.

7.1 Front-End Engineer

Expected Skills

- Advanced React / Next.js
- Form handling and state management
- UX for complex workflows
- API integration

Responsibilities

- Recruiter-facing UI implementation
- Data visualization of shortlists
- Responsive and clean UX

7.2 Back-End Engineer

Expected Skills

- Node.js with TypeScript
- REST API design
- Database modeling (MongoDB)
- Authentication and security basics

Responsibilities

- Business logic implementation
- Data ingestion pipelines
- AI request orchestration

7.3 AI Software Engineer

Expected Skills

- LLM prompt engineering
- Gemini API integration
- Text analysis and ranking logic
- AI explainability concepts

Responsibilities

- Designing the AI decision flow
- Ensuring reliable and interpretable outputs
- Documenting AI assumptions and limitations

7.4 Optional Roles

- Product-minded Engineer
- UI/UX Designer

- DevOps-focused Engineer

8. REFERENCES

1. Umurava AI Hackathon Technical Guide

Full Technical Guide Document — provided by Umurava. Contact: competence@umurava.africa | +250 784 664 612

2. Google Gemini API Documentation

Official documentation for the Gemini API, the mandatory AI/LLM for this project. Available at <https://ai.google.dev/docs>

3. Next.js Documentation

Official documentation for Next.js, the React framework for the frontend. Available at <https://nextjs.org/docs>

4. MongoDB Atlas Documentation

Official documentation for MongoDB Atlas cloud database. Available at <https://www.mongodb.com/docs/atlas/>

5. IEEE Standards for SRS Documentation

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, 1998.

6. Tailwind CSS Documentation

Official documentation for Tailwind CSS. Available at <https://tailwindcss.com/docs>

7. Redux Toolkit Documentation

Official documentation for Redux Toolkit, used for state management. Available at <https://redux-toolkit.js.org/>